

IoTutorMine: A Tool for Mining Hardware Bills of Materials from IoT Tutorial Videos

Abdullah A. Alahmadi[✉]
University of Jeddah
Jeddah, Saudi Arabia
aalahmadi0299.stu@uj.edu.sa

Ahmed O. Bahaj
University of Jeddah
Jeddah, Saudi Arabia
abahaj.stu@uj.edu.sa

Mohammad D. Alahmadi
University of Jeddah
Jeddah, Saudi Arabia
mdalahmadi@uj.edu.sa

Abstract

Internet of Things (IoT) tutorial videos describe the parts a learner must purchase verbally rather than visually, which makes them invisible to frame-based mining techniques developed for programming screencasts. We present IoTutorMine, a tool that recovers structured hardware Bills of Materials (BoMs) from IoT tutorial videos by combining YouTube’s auto-generated transcripts with large language models in a zero-shot setting. IoTutorMine-Web, a public prototype, exposes the pipeline as an interactive IoT tutorial catalog: users paste a YouTube URL to get an on-demand BoM, or browse a pre-indexed library searchable by component (e.g., “every tutorial that uses a DHT11”). We also release a benchmark of 20 manually annotated YouTube tutorials covering Arduino and Raspberry Pi at two difficulty levels, with 131 ground-truth components from 16 distinct creators, together with an expert-curated alias dictionary and evaluation scripts. Under a two-tier entity-resolution strategy that combines syntactic matching with semantic canonicalization, Gemini 3 Flash reaches $F_1 = 0.933$, GPT-5.5 $F_1 = 0.887$, and Claude Opus 4.7 $F_1 = 0.868$, with Gemini significantly outperforming Claude on per-video F_1 (Wilcoxon $p = 0.028$). The results demonstrate that transcript-only BoM extraction is feasible for practical IoT cataloging applications, and that a semantic mapping layer is essential to resolve hardware nomenclature variations.

Video: <https://www.youtube.com/watch?v=1H3GgQFjfZU>

Tool: <https://ahmedbahaj.github.io/IoTutorMine/>

Tool source (DOI): <https://doi.org/10.5281/zenodo.20134482>

Benchmark & replication (DOI): <https://doi.org/10.5281/zenodo.20135165>

CCS Concepts

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

Internet of Things, large language models, video mining, Bill of Materials, tool demonstration

ACM Reference Format:

Abdullah A. Alahmadi, Ahmed O. Bahaj, and Mohammad D. Alahmadi. 2026. IoTutorMine: A Tool for Mining Hardware Bills of Materials from IoT Tutorial Videos. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE ’26)*, October 12–16, 2026, Munich.



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE ’26, Munich, Germany

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2882-2/2026/10

<https://doi.org/10.1145/3832783.3834647>

Germany. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3832783.3834647>

1 Introduction

Internet of Things (IoT) prototyping requires the learner to manage two stacks at once: software running on the device and the hardware on which it runs. Online video tutorials, especially on YouTube, have become a primary resource for IoT learners precisely because they walk through both stacks end to end [10, 13]. A practical question, however, is surprisingly hard to answer at scale: what hardware does this tutorial actually require? Picking the wrong board or missing a sensor can stall a build for days, and today learners answer this question manually by watching candidates and pausing to write down component mentions. A platform-wide index of IoT tutorial BoMs, automatically extracted from the videos themselves, would transform tutorial discovery in the same way that source-code mining transformed how developers navigate software repositories [14].

The software engineering community has built rich tooling for mining programming screencasts, but that tooling assumes the artifact of interest is visually rendered on screen. Existing approaches extract source code [1, 5, 6, 11, 15], XML and metadata [3, 4], UI screens [2], and developer workflows [8, 17] from frames through OCR and computer vision. In IoT tutorials, however, the critical hardware information lives in the audio track, not in the pixels: component names are spoken aloud, the same component appears under many surface forms across creators (“DHT11,” “DHT-11 sensor,” “the temperature module”), and instructors often mention contrast parts that are not in the final build. Methods designed for code-heavy screencasts therefore do not directly transfer to IoT tutorials.

We present IoTutorMine, a tool that fills this gap by mining the spoken narration of IoT tutorials instead of their frames. IoTutorMine is implemented as a transcript-based pipeline driven by a large language model (LLM) in a zero-shot setting, and is exposed to end users through IoTutorMine-Web, a public prototype that turns the extracted BoMs into a searchable IoT tutorial catalog. We also release a benchmark of 20 manually annotated tutorials (131 components, 16 creators) together with a transparent dictionary-based matching procedure, so that other researchers can evaluate new LLMs or new prompts on the same task.

This artefact paper makes three contributions: (i) IoTutorMine-Web, a public IoT tutorial catalog that demonstrates speech-driven BoM extraction in an end-user setting; (ii) a reusable benchmark and evaluation harness for LLM-based IoT video mining; and (iii) empirical baselines for three current LLMs (GPT-5.5, Gemini 3 Flash, Claude Opus 4.7) that future tools can compare against.

2 The IoTutorMine Tool

2.1 Pipeline

IoTutorMine is a three-phase pipeline (Figure 1). *Phase 1: YouTube transcript acquisition.* Given a YouTube IoT tutorial URL, the tool fetches the auto-generated English captions, which are available for the vast majority of public tutorials. We deliberately rely on YouTube’s own transcripts rather than a custom ASR setup so that the pipeline applies directly to any public tutorial at platform scale. *Phase 2: Full transcript input.* The retrieved captions are lowercased and lightly cleaned (timestamps, caption artefacts, and repeated filler tokens are removed), and the transcript is then passed to the model *in full*, without segmentation, chunking, or any retrieval step that would select only part of the narration. This is a deliberate design choice: the transcripts in our corpus (median ≈ 800 words, max $\approx 2,400$) fit comfortably in current LLM context windows, so the model sees the entire narration at once, and component mentions scattered across the introduction, the parts walkthrough, and the assembly description are never truncated away. *Phase 3: Zero-shot LLM extraction.* An LLM is prompted in zero-shot mode — no fine-tuning and no in-context examples — to return a deduplicated markdown table of every “shoppable” component the presenter mentions or uses, labelled as *main* or *alternative*. The prompt explicitly excludes integrated parts (onboard LEDs, internal pull-ups), consumables, software, and components mentioned only for context. The same prompt template is used across LLMs so any difference is attributable to model behaviour rather than to prompt tuning.

2.2 Implementation and Deployment

IoTutorMine-Web is a single-page web application with a stateless backend, designed to make the tool easy to redeploy and easy to extend. The frontend is implemented in Vue.js and is served as a static site from GitHub Pages, which keeps hosting cost zero and removes any custom server-side state. The backend wraps three thin services: a transcript fetcher that calls YouTube’s public caption endpoint, a normalization layer that performs the light cleaning described in Phase 2, and an LLM proxy that forwards the prepared transcript to the configured model with the same zero-shot prompt used in our experiments. Swapping in a new LLM requires changing only the proxy’s API endpoint and the model identifier; the rest of the pipeline is provider-agnostic. The pre-indexed library is a small JSON catalog generated offline by running the same pipeline once per video and caching the predicted BoM, so the catalog mode incurs no LLM cost at query time and remains responsive under load.

2.3 IoTutorMine-Web: The User-Facing Tool

The pipeline is exposed to learners and educators through IoTutorMine-Web at <https://ahmedbahaj.github.io/IoTutorMine/>. Figure 2 shows the landing page. The tool supports two complementary modes of use.

On-demand extraction. The user pastes a YouTube IoT tutorial URL into the input field and clicks *Extract Components*. The backend runs the three-phase pipeline and returns a structured component list rendered next to the video. This makes the tool useful even

for tutorials that are not in the indexed library: a learner deciding whether to follow a specific video can obtain its BoM in seconds without watching the full clip. A typical output for an Arduino weather-station tutorial, for instance, lists Arduino Uno, DHT11, 16x2 LCD, I2C backpack, breadboard, and jumper wires, each tagged as *main* or *alternative* so the learner can immediately see which parts are required and which are substitutes.

Component-based catalog search. The lower panel of Figure 2 shows the pre-indexed library. Each card displays the video thumbnail, title, source channel, and a chip view of the most prominent components in its BoM. The library is queryable by component name through the search field, which turns the catalog into a cross-video index of the form “every tutorial that uses an HC-SR04 ultrasonic sensor.” This is the inverse retrieval problem we set out to solve: instead of starting from a video and listing its parts, the learner starts from a part they already own and finds tutorials that use it.

2.4 Intended Users and Use Cases

IoTutorMine-Web targets three audiences. Learners use it to decide whether a tutorial matches the hardware they already own, or to find tutorials that fit a kit they just bought: pasting a URL replaces the manual watch-and-pause workflow that today is the only way to recover a parts list. Educators use it to assemble parts lists for lab assignments and to filter tutorials by platform and component before recommending them to students. Researchers use the underlying pipeline and benchmark as a baseline for new LLMs, new prompts, or extensions to additional IoT platforms (e.g., ESP32, micro:bit). Because the predicted BoMs are exposed directly next to the video thumbnails on the landing page, extraction errors are visible to anyone visiting the site, which makes the tool self-auditing rather than a black box.

3 Dataset and Evaluation Harness

3.1 Benchmark

To evaluate the pipeline described above, we release a benchmark of 20 IoT tutorial videos collected from YouTube under four criteria: English-language tutorials that build an Arduino or Raspberry Pi prototype, with available transcripts, a clearly identifiable verbal parts list, and distinct creators where possible. The benchmark covers two platforms (Arduino, Raspberry Pi) and two difficulty levels (beginner, intermediate). The corpus contains 131 manually annotated components across 16 distinct channels, with per-video duration from 3.18 to 14.08 minutes and 4 to 10 components per video (median 6). Each annotation records the canonical component name, the surface form used in the video, the quantity if stated, the source signal (verbal, visual, or both), and a usage status (USED, ALTERNATIVE, OPTIONAL, or MENTIONED_NOT_USED). Mechanical accessories bundled with another part (e.g., a stylus shipped with a touch display) and components mentioned only as contrast are excluded from the BoM.

3.2 Matching Procedure

Evaluating hardware extraction is challenging due to nomenclature fragmentation (e.g., “1602” vs. “16x2 LCD”). To address this, we implemented a two-tier Entity Resolution strategy. The first tier

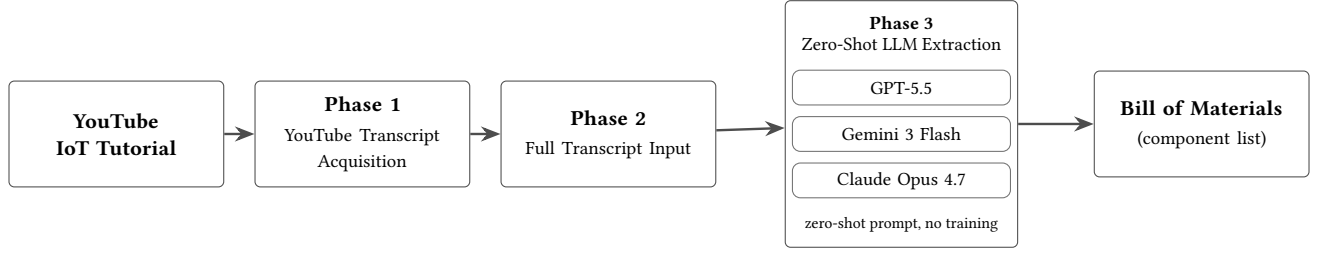


Figure 1: The IoTutorMine pipeline. A single YouTube IoT tutorial URL is converted into a structured hardware BoM: Phase 1 acquires the tutorial’s YouTube transcript, Phase 2 passes the full transcript to the model without segmentation, and Phase 3 performs zero-shot LLM extraction. The resulting BoM is rendered next to the video in IoTutorMine-Web.

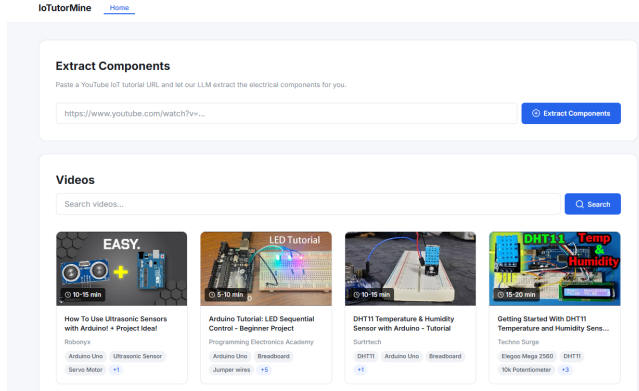


Figure 2: The IoTutorMine-Web prototype. The top panel takes a YouTube URL and runs on-demand BoM extraction. The bottom panel is a pre-indexed library where every video is annotated with searchable component tags, enabling cross-video retrieval by hardware part.

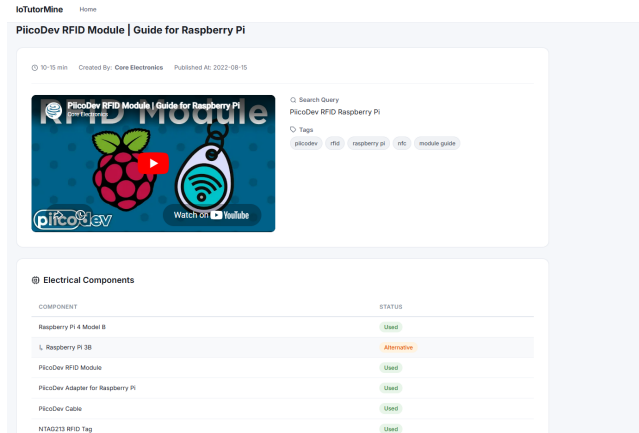


Figure 3: Per-tutorial detail view: the embedded YouTube player alongside the extracted BoM, with each component tagged as used or alternative.

is a Syntactic Layer that performs automated substring matching

Table 1: Component extraction quality at the corpus level (micro) and per-video level (macro mean $\pm$ std, $N = 20$).

Model	Corpus (micro)			Per-video (macro)		
	P	R	F_1	P	R	F_1
GPT-5.5	0.905	0.870	0.887	0.88	0.84	0.85
Gemini 3 Flash	0.907	0.962	0.933	0.89	0.90	0.89
Claude Opus 4.7	0.882	0.855	0.868	0.87	0.84	0.85

to handle minor variations. The second tier is a Semantic Normalization Layer driven by an expert-curated Alias Dictionary of 45 canonical entries and 106 variant mappings. This dictionary maps technically distinct surface forms to a single “shoppable” identity; for instance, it unifies 1602A, LCD1602, and 16x2 Display under the canonical 16x2 LCD Module. For families whose variants are commercially interchangeable from a learner’s perspective (resistor values, LED colours, RFID tag protocols), the dictionary further applies a parent canonicalization policy: all resistor values collapse to Resistor, colored LEDs to LED, and RFID tag protocols to RFID Tag. Physically distinct variants (microcontroller versions, sensor part numbers, display sizes) are kept strictly separate. We also enforced a strict Specificity Check that rejects generic matches (e.g., Raspberry Pi) against specific ground-truth models (e.g., Raspberry Pi 5), penalizing models that fail to capture critical version details. We deliberately avoided fuzzy or embedding-based similarity tiers on top, as a similarity threshold introduces an arbitrary cutoff and can quietly accept near-misses that are not in fact the same component. The full dictionary and matching script are released as part of the replication package so that other researchers can audit, extend, or override the policy.

3.3 Empirical Results

We benchmark three state-of-the-art LLMs under identical zero-shot conditions, each queried in a fresh, isolated session of its provider’s web interface to eliminate cross-query carry-over. Table 1 summarizes the results. Gemini 3 Flash achieves the strongest performance with corpus-level $F_1 = 0.933$ (precision 0.907, recall 0.962), recovering 126 of 131 ground-truth components. GPT-5.5 ($F_1 = 0.887$) and Claude Opus 4.7 ($F_1 = 0.868$) follow closely, with precision similar across all three models (0.882–0.907). A Wilcoxon signed-rank test on per-video F_1 shows that Gemini significantly

outperforms Claude ($W = 1.0$, $p = 0.028$); the other pairwise contrasts are not significant at the 0.05 level. Both Gemini and Claude reach perfect F_1 on the Arduino intermediate subset, indicating that once narration becomes structured and explicit the extraction problem is close to solved for current LLMs.

A manual inspection of all 40 false positives groups them into five categories: generic specificity errors (24 cases, 60%) where the model returns a generic name when the ground truth is specific (Raspberry Pi for a Raspberry Pi 5 build, or bare LED when the project uses a specific colour); mentioned-not-used errors (10%) where the model extracts a contrast part that the presenter only compares against; compound extractions (10%) such as “LCD 1602 Module with I2C” which conflate two physically separate components; mechanical accessories (10%) like a stylus or mounting board that ship bundled but are not separately purchasable; and pure hallucinations (10%) where the model produces a component that does not appear in the transcript. The dominance of specificity errors is encouraging because they are addressable through better prompting and downstream post-processing, whereas hallucinations would be much harder to suppress. We also found that difficulty level affects extraction quality more than platform does: pooling across models, intermediate tutorials yield significantly higher F_1 than beginner tutorials ($U = 179.0$, $p = 0.0003$), while the Arduino vs. Raspberry Pi gap is not significant ($p = 0.91$).

4 Related Tools

IoTutorMine sits next to two lines of tools. The first is frame-based mining of programming screencasts. CodeTube [14] and Codemotion [8] index tutorial fragments and edit intervals; psc2code [6] and CodeT5-OCRfix [11] recover source code from frames through OCR with temporal denoising; CodeSCAN [12] provides a benchmark for screencast analysis; VID2META [3] and VID2XML [4] extend extraction to Java metadata and XML; frame-based summarization condenses a screencast into representative key frames [9]; ActionNet [16] and SeeHow [17] reconstruct coding workflows from observed UI actions. All of these tools assume that the artefact is visually present in the frame and can be recovered through OCR or computer vision. IoTutorMine targets the opposite case: the artefact lives in the audio. The second line is text- and transcript-based analysis of developer videos [7], which has produced tags and summaries but not structured hardware artefacts. To our knowledge, IoTutorMine is the first tool to mine a typed, deduplicated hardware BoM from IoT tutorial speech. The two lines are complementary rather than competing: future versions of IoTutorMine could fuse transcript-derived component names with frame-derived wiring detail to produce a richer multimodal artefact.

5 Availability and Reuse

IoTutorMine-Web is openly accessible at <https://ahmedbahaj.github.io/IoTutorMine/> and its source is archived on Zenodo at <https://doi.org/10.5281/zenodo.20134482> under the MIT licence. The benchmark and full replication package (videos metadata, transcripts, ground-truth annotations, LLM outputs, matching dictionary, and evaluation scripts) is separately archived on Zenodo at <https://doi.org/10.5281/zenodo.20135165> under the CC-BY-4.0 licence. A 5-minute screencast demonstrating the catalog

and component search modes of IoTutorMine-Web is available at <https://www.youtube.com/watch?v=1H3GgQFjfZU>. The pipeline is provider-agnostic by design: swapping in a new LLM requires changing only an API endpoint, and adding new IoT platforms requires only extending the matching dictionary. We see two natural extensions for community contribution. First, the benchmark can grow to additional platforms (ESP32, micro:bit), advanced and expert tutorials, and other languages, where Arabic-language IoT tutorials are an immediate priority for our group given the 400-million-strong Arabic-speaking learner population. Second, the matching dictionary is a community artefact: pull requests that add new aliases or new specificity tokens will directly improve evaluation fidelity for future LLMs evaluated on the same benchmark.

Data Availability Statement

The IoTutorMine-Web source code is archived on Zenodo at <https://doi.org/10.5281/zenodo.20134482> under the MIT license. The benchmark and full replication package (video metadata, transcripts, ground-truth annotations, LLM outputs, the alias dictionary, and evaluation scripts) is archived on Zenodo at <https://doi.org/10.5281/zenodo.20135165> under the CC-BY-4.0 license. A five-minute demonstration screencast is available at <https://www.youtube.com/watch?v=1H3GgQFjfZU>.

References

- [1] Mohammad Alahmadi, Abdulkarim Khormi, Biswas Parajuli, Jonathan Hassel, Sonia Haiduc, and Piyush Kumar. 2020. Code Localization in Programming Screencasts. *Empirical Software Engineering* 25, 2 (2020), 1536–1572. doi:10.1007/s10664-019-09759-w
- [2] Mohammad Alahmadi, Abdulkarim Malkadi, and Sonia Haiduc. 2020. UI Screens Identification and Extraction from Mobile Programming Screencasts. In *Proceedings of the 28th International Conference on Program Comprehension (ICPC)*. ACM, 319–330. doi:10.1145/3387904.3389265
- [3] Mohammad D. Alahmadi. 2022. VID2META: Complementing Android Programming Screencasts with Code Elements and GUIs. *Mathematics* 10, 17 (2022), 3175. doi:10.3390/math10173175
- [4] Mohammad D. Alahmadi. 2023. VID2XML: Automatic Extraction of a Complete XML Data from Mobile Programming Screencasts. *IEEE Transactions on Software Engineering* 49, 4 (2023), 1726–1740. doi:10.1109/TSE.2022.3188898
- [5] Mohammad D. Alahmadi and Moayad Alshangiti. 2024. Optimizing OCR Performance for Programming Videos: The Role of Image Super-Resolution and Large Language Models. *Mathematics* 12, 7 (2024), 1036. doi:10.3390/math12071036
- [6] Lingfeng Bao, Zhenchang Xing, Xin Xia, David Lo, Minghui Wu, and Xiaohu Yang. 2020. psc2code: Denoising Code Extraction from Programming Screencasts. *ACM Transactions on Software Engineering and Methodology* 29, 3, Article 21 (2020). doi:10.1145/3392093
- [7] Javier Escobar-Avila, Esteban Parra, and Sonia Haiduc. 2017. Text Retrieval-Based Tagging of Software Engineering Video Tutorials. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 341–343. doi:10.1109/ICSE-C.2017.121
- [8] Kandarp Khandwala and Philip J. Guo. 2018. Codemotion: Expanding the Design Space of Learner Interactions with Computer Programming Tutorial Videos. In *Proceedings of the Fifth Annual ACM Conference on Learning at Scale*. ACM, 1–10. doi:10.1145/3231644.3231652
- [9] F. Khoshnoud, Mohammad R. Moosavi, and N. Azouji. 2025. Summarizing Mobile Programming Screencasts. *Signal, Image and Video Processing* 19 (2025), 866. doi:10.1007/s11760-025-04503-y
- [10] Laura MacLeod, Margaret-Anne Storey, and Andreas Bergen. 2015. Code, Camera, Action: How Software Developers Document and Share Program Knowledge Using YouTube. In *Proceedings of the 23rd IEEE International Conference on Program Comprehension (ICPC)*. IEEE, 104–114. doi:10.1109/ICPC.2015.19
- [11] Abdulkarim Malkadi, Ahmad Tayeb, and Sonia Haiduc. 2023. Improving Code Extraction from Coding Screencasts Using a Code-Aware Encoder-Decoder Model. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1492–1504. doi:10.1109/ASE56229.2023.00184
- [12] Alexander Naumann, Felix Hertlein, Jacqueline Höllig, Lucas Cazzonelli, and Steffen Thoma. 2024. CodeSCAN: ScreenCast ANalysis for Video Programming Tutorials. arXiv:2409.18556 [cs.LG] doi:10.48550/arXiv.2409.18556

- [13] Luca Ponzanelli, Gabriele Bavota, Andrea Mocci, Massimiliano Di Penta, Rocco Oliveto, Mir Hasan, Barbara Russo, Sonia Haiduc, and Michele Lanza. 2016. Too Long; Didn't Watch! Extracting Relevant Fragments from Software Development Video Tutorials. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 261–272. doi:10.1145/2884781.2884824
- [14] Luca Ponzanelli, Gabriele Bavota, Andrea Mocci, Massimiliano Di Penta, Rocco Oliveto, Barbara Russo, Sonia Haiduc, and Michele Lanza. 2016. CodeTube: Extracting Relevant Fragments from Software Development Video Tutorials. In *Proceedings of the 38th International Conference on Software Engineering Companion (ICSE-C)*. ACM, 645–648. doi:10.1145/2889160.2889172
- [15] Shir Yadid and Eran Yahav. 2016. Extracting Code from Programming Tutorial Videos. In *Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward')*. ACM, 98–111. doi:10.1145/2986012.2986021
- [16] Dehai Zhao, Zhenchang Xing, Chunyang Chen, Xin Xia, and Guoqiang Li. 2019. ActionNet: Vision-Based Workflow Action Recognition from Programming Screencasts. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*. IEEE, 350–361. doi:10.1109/ICSE.2019.00049
- [17] Dehai Zhao, Zhenchang Xing, Xin Xia, Deheng Ye, Xiwei Xu, and Liming Zhu. 2023. SeeHow: Workflow Extraction from Programming Screencasts through Action-Aware Video Analytics. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1946–1957. doi:10.1109/ICSE48619.2023.00165

Received 2026-05-12; accepted 2026-06-19